SIMATS ENGINEERING

ASSESSMENT TOOL 2 - MEDIUM (Class Test)

COURSE CODE: CSA09 **COURSE NAME: Programming in Java**

COURSE OUTCOME COVERED

CO3: *Build multithreaded Java programs by applying thread priorities, synchronisation, and inter-thread communication mechanisms to achieve concurrent program execution and use exception handling techniques (BL3,BL4)*

Assessment Tool Weightage:20%

QUESTIONS: 10 **Total Marks: 50**

ANNEXURE A

CLASS TEST

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

ANNEXURE

CLASS TEST

1. Write a Java program to create two threads using Runnable. Assign different priorities and observe execution order. Analyze whether priority affects scheduling consistently. **BTL:** BL3
2. Develop a program where multiple threads access a shared counter. Use synchronized methods to prevent race conditions. Analyze the effect of removing synchronization. **BTL:** BL4
3. Implement Producer and Consumer threads using a shared buffer. Apply wait() and notify() for coordination. Analyze how improper synchronization leads to deadlock.
   BTL: BL4
4. Create a multithreaded program that handles runtime exceptions. Use try-catch blocks inside threads. Evaluate impact of unhandled exceptions on thread execution.
   BTL: BL3
5. Write a program with three threads having different priorities. Perform CPU-intensive tasks and observe execution. Analyze limitations of priority scheduling in Java.
   BTL: BL4
6. Create two threads competing for two shared resources. Demonstrate a deadlock situation using synchronization.Suggest methods to avoid deadlock.
   BTL: BL4
7. Design a program where one thread produces data and another consumes it. Use wait() and notifyAll() methods. Analyze communication efficiency and delays.
   BTL: BL3
8. Develop a multithreaded program to read/write files concurrently. Use synchronization to avoid data corruption. Handle file-related exceptions effectively.
   BTL: BL3
9. Write a program where two threads update the same variable. Show inconsistent results without synchronization. Analyze how synchronization resolves the issue.
   BTL: BL4

10. Create a program demonstrating thread lifecycle states. Introduce exceptions during execution and handle them. Analyze how exceptions affect thread termination.
BTL: BL4

11. Write a program to create two threads using Runnable. Assign names and start them. Observe execution order and explain results.

12. Create two threads by extending Thread. Compare behavior with Runnable approach.Analyze advantages of both methods.

13. Create multiple threads updating a shared counter. Observe inconsistent outputs. Explain race condition.

14. Use synchronized method to fix race condition. Compare output with unsynchronized version. Analyze correctness.

15. Create threads using sleep() method. Observe execution delays. Analyze scheduling behavior.

16. Create threads and use join() to control execution order. Analyze impact on synchronization.

17. Implement two threads using wait() and notify(). Analyze coordination and delays.

18. Extend producer-consumer with multiple consumers. Use notifyAll(). Analyze efficiency.

19. Create two threads locking resources in reverse order. Demonstrate deadlock. Suggest prevention.

20. Modify previous program to avoid deadlock. Use ordering or timeout. Analyze results.

21. Assign different priorities to threads. Run CPU tasks. Analyze execution order.

22. Compare synchronized block and method. Analyze performance and flexibility.

23. Multiple threads read/write file. Use synchronization. Analyze data consistency.

24. Throw exceptions inside threads. Handle using try-catch. Analyze behavior.

25. Allow thread to throw unhandled exception. Observe impact on program.
26. Demonstrate NEW, RUNNABLE, WAITING states. Analyze transitions.
27. Compare Runnable and Callable. Implement both. Analyze return values.
28. Use ExecutorService to manage threads. Analyze performance improvement.
29. Split array among threads. Calculate sum concurrently. Compare with sequential.
30. Multiple threads access shared object. Use locks. Analyze synchronization.
31. Replace synchronized with ReentrantLock. Compare flexibility and performance.
32. Simulate thread starvation. Analyze cause and prevention.
33. Implement bounded buffer. Use wait/notify. Analyze blocking.
34. Measure delay between producer and consumer. Analyze bottlenecks.
35. Divide matrix rows among threads. Analyze performance.
36. Simulate transactions across accounts. Ensure consistency.
37. Multiple ATMs access same account. Analyze synchronization issues.
38. Implement reader-writer scenario. Allow multiple readers. Analyze locking.
39. Use synchronized collections. Compare with normal collections.
40. Demonstrate volatile variable. Analyze visibility issues.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand